3 - Funciones de orden superior

Tabla de contenidos

1 Introducción 1
2 Pilares fundamentales 1
2.1 *Map* 2
2.1.a `map` con datos complejos 3
2.1.b `map` con múltiples iterabes 5
2.2 *Filter* 6
2.3 *Reduce* 8
2.4 Resumen 10
2.4.a *Map* y *filter* como casos particulares de *reduce* 😱 11
3 *Comprehensions* 12
3.1 *Comprehension* como reemplazo de *map* 12
3.2 *Comprehension* como reemplazo de *filter* 13
4 Evaluación parcial de funciones 16
5 Decoradores 19
5.1 Decoradores que reciben argumentos 21
5.2 Decoradores que devuelven valores 22
5.3 Azúcar sintáctico 22
5.4 Ejemplo: medir tiempo de ejecución 23

1 Introducción

Las funciones de orden superior son una herramienta muy importante en la programación funcional. A lo largo de esta unidad, trabajaremos con las siguientes variedades de funciones de orden superior:

- Funciones que aceptan funciones como argumentos.
- Funciones que devuelven una función como resultado.
- Funciones que aceptan funciones como argumentos y devuelven una función como resultado.

En este capítulo comenzamos enfocándonos en las funciones de orden superior más elementales: `map`, `filter` y `reduce`; todas ellas reciben funciones como argumentos. Luego, aprenderemos sobre las *comprehensions*, que constituyen la alternativa moderna y Pythonica a las funciones mencionadas anteriormente. Finalmente, trabajaremos con funciones que devuelven funciones cuando exploremos evaluación parcial de funciones y el uso de decoradores.

2 Pilares fundamentales

Las funciones `map`, `filter` y `reduce` son funciones de orden superior fundamentales en la programación funcional. Actúan como primitivas básicas para procesar y transformar secuencias,

y muchas otras operaciones funcionales pueden construirse a partir de ellas o expresarse en términos de estas.

Las primeras dos, `map` y `filter`, están disponibles por defecto en nuestra sesión de Python (ya que son funciones *built-in*), mientras que a `reduce` la tenemos que importar desde el módulo estándar `functools`.

2.1 *Map*

Supongamos que tenemos una secuencia de palabras y queremos invertir el orden de los caracteres de cada una. Para ello, vamos a rebanadar cada cadena desde el principio al final usando un paso de `-1`. Por ejemplo:

```
"cosa"[::-1]
```

```
'asoc'
```

Si quisiéramos obtener una lista con las palabras invertidas, podríamos crear una nueva lista, recorrer la original con un bucle `for`, invertir cada palabra y guardarla en la lista nueva.

```
palabras = ["hola", "mate", "somos"," libro", "conocer", "anilina",
"programa"]
palabras_invertidas = []

for palabra in palabras:
    palabras_invertidas.append(palabra[::-1])

print("Palabras originales:", palabras, "\n", sep="\n")
print("Palabras invertidas:", palabras_invertidas, sep="\n")
```

```
Palabras originales:
['hola', 'mate', 'somos', ' libro', 'conocer', 'anilina', 'programa']


Palabras invertidas:
['aloh', 'etam', 'somos', 'orbil ', 'reconoc', 'anilina', 'amargorp']
```

La alternativa funcional consiste en utilizar `map` para **aplicar una función a cada palabra de la secuencia**. En este caso, aplicamos la función `invertir`, que invierte los caracteres de una palabra, a cada elemento de la lista `palabras`.

```
def invertir(x):
    return x[::-1]

list(map(invertir, palabras))
```

```
['aloh', 'etam', 'somos', 'orbil ', 'reconoc', 'anilina', 'amargorp']
```

Así, se obtiene una nueva lista con las palabras invertidas, sin necesidad de iterar manualmente con un bucle `for`.

Si quisiéramos que el programa fuese aún más conciso, podríamos usar una función anónima en vez de una función regular:

```
list(map(lambda x: x[::-1], palabras))
```

```
['aloh', 'etam', 'somos', 'orbil ', 'reconoc', 'anilina', 'amargorp']
```

> ℹ El objeto `map` 🔍
>
> En el ejemplo anterior usamos `list` para convertir el resultado de `map` en una lista. Este paso, que puede parecer innecesario, es fundamental si queremos obtener una lista como resultado final. De lo contrario, la llamada a `map` devuelve un objeto de tipo `map`.
>
> ```
> map(lambda x: x[::-1], palabras)
> ```
>
> ```
> <map object at 0x7fd2fc1ad360>
> ```
>
> Este objeto, **perezoso** e **iterable**, puede recorrerse o convertirse en otras colecciones como listas, tuplas o conjuntos:
>
> ```
> list(map(lambda x: x[::-1], palabras))
> ```
>
> ```
> ['aloh', 'etam', 'somos', 'orbil ', 'reconoc', 'anilina', 'amargorp']
> ```

2.1.a `map` con datos complejos

En el ejemplo anterior se usó `map` sobre una secuencia simple de cadenas de texto. Sin embargo, eso no implica que su uso se limite a casos sencillos.

Supongamos ahora que tenemos una lista anidada de números, es decir, una lista que contiene otras listas con valores numéricos:

```
ventas = [
    [22.5, 9.3, 11.0],
    [5.4, 22.5],
    [3.0, 3.0, 12.9, 7.5],
]
```

Si queremos calcular el total de cada sublista, podemos combinar `map` con la función `sum`. Esto aplica `sum` a cada elemento de la lista `ventas`, generando como resultado una nueva lista con los totales de cada sublista.

```
list(map(sum, ventas))
```

```
[42.8, 27.9, 26.4]
```

De manera similar, se puede obtener el mínimo, el máximo, la media u otra medida de interés aplicando la función correspondiente a cada sublista.

Usando una combinación más compleja de `map`s y expresiones `lambda`, se puede determinar cuáles sublistas de `ventas` contienen al menos un valor mayor a 20.

```
list(
    map(
        lambda sublista: any(map(lambda x: x > 20,  sublista)),
        ventas
    )
)
```

```
[True, True, False]
```

Como puede observarse, un programa que utiliza `map` junto con expresiones `lambda` puede volverse difícil de leer y comprender rápidamente, especialmente a medida que la lógica se vuelve más compleja.

Para finalizar este listado de ejemplos, observemos uno donde se crea un diccionario a partir del `map`, en vez de una lista.

Se cuenta con una lista de diccionarios. Cada diccionario contiene el nombre y las calificaciones de una persona. Nuestro objetivo es obtener un nuevo diccionario que tenga por claves al nombre de la persona, y por valor a la nota promedio.

```
notas = [
    {
        "nombre": "Mariano",
        "notas": [6, 9, 9, 8]
    },
    {
        "nombre": "Daniela",
        "notas": [6, 7, 7, 8]
    },
    {
        "nombre": "Sofía",
        "notas": [8, 6, 9, 8]
```

```
    },
]
```

Sin utilizar un enfoque funcional, una solución posible es la siguiente:

```
def media(x):
    return sum(x) / len(x)

promedios = {}

for datum in notas:
    promedios[datum["nombre"]] = media(datum["notas"])

promedios
```

```
{'Mariano': 8.0, 'Daniela': 7.0, 'Sofía': 7.75}
```

En cambio, utilizando `map`:

```
dict(map(lambda datum: (datum["nombre"], media(datum["notas"])), notas))
```

```
{'Mariano': 8.0, 'Daniela': 7.0, 'Sofía': 7.75}
```

La clave está en notar que la expresión `lambda` **devuelve una tupla de dos elementos**, donde el primero es el nombre y el segundo, la nota promedio. A partir de estos pares `(str, float)`, `dict` puede construir directamente un diccionario con los `str` en las claves y los `float` en los valores.

2.1.b `map` con múltiples iterabes

Hasta ahora hemos utilizado `map` con funciones que se aplican sobre los elementos de un único iterable. Sin embargo, `map` también acepta múltiples iterables y los recorre en paralelo, lo que la convierte en una **función variádica**. De este modo, se puede usar `map` para aplicar funciones que toman más de un argumento.

Supongamos que queremos redondear un listado de números utilizando diferentes niveles de precisión. Para redondear un único número podemos usar directamente `round`:

```
round(29.12951138, 4)
```

```
29.1295
```

Si quisiéramos redonear múltiples números en una lista, usando el mismo nivel de precisión, podemos usar `map` y `round`:

```
numeros = [
    30.60726375,
    78.12297368,
    61.94972186,
    68.78842783,
    55.60016942,
    94.9760221,
    90.41151716,
    38.72727347,
    21.30193307,
    66.39407577
]
list(map(lambda x: round(x, 3), numeros))
```

```
[30.607, 78.123, 61.95, 68.788, 55.6, 94.976, 90.412, 38.727, 21.302, 66.394]
```

¿Y si quisiéramos aplicar diferentes niveles de precisión a cada número? Para ello, **también podemos usar `map`**. Definimos una función que reciba dos argumentos y luego iteramos en paralelo sobre dos iterables: uno con los números y otro con las precisiones correspondientes.

```
precisiones = [2, 2, 3, 3, 4, 4, 5, 5, 2, 2]
list(map(lambda x, y: round(x, y), numeros, precisiones))
```

```
[30.61, 78.12, 61.95, 68.788, 55.6002, 94.976, 90.41152, 38.72727, 21.3,
66.39]
```

i ¿Qué pasa si un iterable es más corto que el otro? 🤔

Cuando se recorren múltiples iterables con `map`, la iteración se detiene tan pronto como se agota el iterable más corto. Por ejemplo, si tenemos 10 números pero solo 5 precisiones, `map` aplicará la función únicamente a los primeros 5 pares de elementos:

```
precisiones = [1, 2, 3, 4, 5]
list(map(lambda x, y: round(x, y), numeros, precisiones))
```

```
[30.6, 78.12, 61.95, 68.7884, 55.60017]
```

2.2 *Filter*

`filter` se utiliza para seleccionar —o, más precisamente, filtrar— elementos de un iterable según el resultado de aplicar una función. A diferencia de `map`, la función usada por `filter` se aplica sobre los elementos de **un solo iterable** y **debe devolver un valor *booleano***. El resultado es un nuevo iterable que contiene únicamente los elementos para los que la función retorna `True`.

Como ejemplo del uso de `filter`, vamos a seleccionar las notas menores a 6 a partir de una lista de calificaciones.

```
notas = [6, 9, 6, 5, 7, 4, 5, 8, 3, 10, 9, 4, 7, 8]
list(filter(lambda x: x < 6, notas))
```

```
[5, 4, 5, 3, 4]
```

De este modo, resulta sencillo calcular el promedio de las notas de aquellos que no aprobaron:

```
media(list(filter(lambda x: x < 6, notas)))
```

```
4.2
```

Retomando el ejemplo del listado de palabras que se querían invertir, se podría usar `filter` para seleccionar solo aquellas palabras que sean palíndromos, es decir, capicúa.

```
palabras = ["hola", "mate", "somos"," libro", "conocer", "anilina",
"programa"]
capicuas = list(filter(lambda p: p[::-1] == p, palabras))
capicuas
```

```
['somos', 'anilina']
```

Naturalmente, `filter` también puede utilizarse para filtrar objetos más complejos. Por ejemplo, si tenemos una lista de diccionarios con información de estudiantes (nombre, ciudad de origen, edad y fecha de inscripción), podemos usar `filter` para seleccionar aquellos que cumplan **una o más condiciones**. En ese caso, el valor *booleano* que devuelve la función se construye combinando condiciones mediante operadores lógicos como `and`.

```
datos = [
    {"nombre": "Agustina", "ciudad": "Casilda", "edad": 18, "inscripcion":
2025},
    {"nombre": "Emiliano", "ciudad": "Rosario", "edad": 21, "inscripcion":
2024},
    {"nombre": "David", "ciudad": "Pergamino", "edad": 19, "inscripcion":
2024},
    {"nombre": "Julieta", "ciudad": "Rosario", "edad": 19, "inscripcion":
2025},
    {"nombre": "Victoria", "ciudad": "Chañar Ladeado", "edad": 18,
"inscripcion": 2025},
    {"nombre": "Fernando", "ciudad": "Rosario", "edad": 20, "inscripcion":
2024},
    {"nombre": "Mateo", "ciudad": "Pérez", "edad": 23, "inscripcion": 2025},
```

```
    {"nombre": "Lucía", "ciudad": "Rosario", "edad": 22, "inscripcion": 2022},
    {"nombre": "Joaquín", "ciudad": "Casilda", "edad": 19, "inscripcion":
2025},
    {"nombre": "Micaela", "ciudad": "Rosario", "edad": 18, "inscripcion":
2024},
]

list(filter(lambda x: x["ciudad"] == "Rosario" and x["inscripcion"] == 2025,
datos))
```

```
[{'nombre': 'Julieta', 'ciudad': 'Rosario', 'edad': 19, 'inscripcion': 2025}]
```

2.3 *Reduce*

La función `reduce` permite **reducir** una secuencia a un único valor aplicando de forma sucesiva una función de dos argumentos sobre sus elementos.

Para utilizarla, es necesario importarla desde el módulo estándar `functools`:

```
from functools import reduce
```

`reduce` aplica la función acumulando resultados de a pares, desde el primer elemento hasta el último. Por ejemplo:

```
reduce(lambda x, y: x + y, [1, 2, 3, 4, 5])
```

equivale a:

```
((((1 + 2) + 3) + 4) + 5)
```

En este caso, es simplemente una forma más rebuscada de escribir `sum([1, 2, 3, 4, 5])` en Python.

Para entender cómo funciona el proceso de acumulación en `reduce`, podemos definir una función que imprima los valores de sus argumentos en cada paso:

```
def sumar(x, y):
    print(f"x={x}, y={y}")
    return x + y

reduce(sumar, [1, 2, 3, 4, 5])
```

```
x=1, y=2
x=3, y=3
x=6, y=4
```

```
x=10, y=5
15
```

En la primera llamada, `x` e `y` son los dos primeros elementos de la secuencia. En la segunda, `x` es el resultado de la llamada anterior, e `y` es el siguiente elemento de la secuencia. Este proceso continúa hasta que se recorre toda la lista. En resumen:

- `x` representa el valor acumulado hasta el momento, e
- `y` es el nuevo elemento a combinar.

Así, `reduce` va aplicando la función paso a paso, acumulando resultados hasta obtener un único valor final.

Muchas operaciones comunes, como sumas, productos, mínimos o máximos, pueden expresarse mediante reducciones. Por ejemplo, es posible calcular el factorial de un número utilizando una `reduce`:

```
def factorial(n):
    return reduce(lambda x, y: x * y, range(1, n + 1))

factorial(5)
```

```
120
```

La reducción mediante la multiplicación de dos números, aplicada a la secuencia del 1 al `n`, da como resultado el factorial de `n`.

Finalmente, podemos ver que combinando una función que devuelve el mayor de dos números y una reducción, es posible obtener el máximo de una secuencia.

```
def mayor(x, y):
    if x > y:
        return x
    return y

reduce(mayor, [23, 49, 6, 32, 101, 9])
```

```
101
```

Vale la pena mencionar que `reduce` acepta un tercer argumento opcional, que especifica el valor inicial de la reducción. Este valor se utiliza como punto de partida antes de procesar los elementos del iterable.

```
def sumar(x, y):
    print(f"x={x}, y={y}")
```

```
    return x + y

reduce(sumar, [1, 2, 3, 4, 5], 20)
```

```
x=20, y=1
x=21, y=2
x=23, y=3
x=26, y=4
x=30, y=5
35
```

ℹ Expresiones condicionales 🔀 😱

La reducción anterior puede expresarse de forma más concisa utilizando **expresiones condicionales**:

```
reduce(lambda x, y: x if x > y else y, [23, 49, 6, 32, 101, 9])
```

Estas expresiones permiten simplificar asignaciones condicionales. Por ejemplo, el siguiente bloque:

```
if x > y:
    valor = x
else:
    valor = y
```

puede escribirse de manera más compacta así:

```
valor = x if x > y else y
```

En términos generales, la sintaxis es:

```
<valor_si_verdadero> if <condición> else <valor_si_falso>
```

2.4 Resumen

El siguiente bloque de código resume el funcionamiento de `map`, `filter` y `reduce`.

```
numeros = [1, 2, 3, 4, 5]

# Map: Aplicar una función a cada elemento de un iterable
cuadrados = list(map(lambda x: x**2, numeros))
print("Map [cuadrados]:", cuadrados)
```

```
# Filter: Devuelve el subconjunto de elementos para los que la función
devuelve True
pares = list(filter(lambda x: x % 2 == 0, numeros))
print("Filter [pares]:", pares)

# Reduce: Aplica una función de dos argumentos de manera acumulativa a los
elementos de una secuencia
producto = reduce(lambda x, y: x * y, numeros)
print("Reduce [producto]:", producto)
```

```
Map [cuadrados]: [1, 4, 9, 16, 25]
Filter [pares]: [2, 4]
Reduce [producto]: 120
```

2.4.a *Map* y *filter* como casos particulares de *reduce* 😱

Por otro lado, algo menos evidente es que tanto `map` como `filter` pueden verse como casos particulares de `reduce`.

Esta aplicación de `map`:

```
list(map(lambda x: x * 2, range(10)))
```

```
[0, 2, 4, 6, 8, 10, 12, 14, 16, 18]
```

Puede ser reproducida con el siguiente uso de `reduce`:

```
def dup(x):
    return x * 2

reduce(lambda seq, x: seq + [dup(x)], range(10), [])
```

```
[0, 2, 4, 6, 8, 10, 12, 14, 16, 18]
```

Y el siguiente uso de `filter`

```
list(filter(lambda x: x % 2 == 1, range(10)))
```

```
[1, 3, 5, 7, 9]
```

Se puede expresar también con `reduce`:

```python
def es_impar(x):
    return x % 2 == 1

reduce(lambda seq, x: seq + [x] if es_impar(x) else seq, range(10), [])
```

```
[1, 3, 5, 7, 9]
```

Estas expresiones con `reduce()` son complejas, pero ilustran claramente el poder de la función: cualquier operación que pueda definirse a partir de una combinación sucesiva de elementos puede, al menos en principio, expresarse como una reducción, aunque no siempre sea la forma más clara o recomendada de hacerlo.

3 *Comprehensions*

Cuando usamos `map` y `filter` obtenemos objetos especiales: `map` devuelve un objeto de tipo `map`, y `filter` devuelve un objeto de tipo `filter`. Estos objetos son iterables y perezosos, lo que significa que no realizan ninguna operación hasta que se los recorre o convierte en una colección, como una lista. Por eso, si queremos ver directamente el resultado de una transformación o filtrado, necesitamos envolverlos con `list()`:

```python
numeros = [1, 2, 3]
list(map(lambda x: x * 2, numeros))          # → [2, 4, 6]
list(filter(lambda x: x % 2 == 0, numeros))  # → [2]
```

Aunque `map` y `filter` siguen siendo completamente válidos y útiles, hoy en día se consideran formas anticuadas o menos idiomáticas de construir listas transformadas o filtradas en Python.

La alternativa moderna y, en general preferida, son las **comprensiones de listas** (del inglés, *list comprehensions*), que permiten expresar las mismas ideas de forma más clara y legible:

```python
numeros = list(range(11))
[x * 2 for x in numeros] # Reemplaza a list(map(...))
```

```
[0, 2, 4, 6, 8, 10, 12, 14, 16, 18, 20]
```

```python
[x for x in numeros if x % 2 == 0] # Reemplaza a list(filter(...))
```

```
[0, 2, 4, 6, 8, 10]
```

3.1 *Comprehension* como reemplazo de *map*

Supongamos que tenemos una lista de números y queremos restarles su media.

Una forma de hacerlo utilizando un bucle `for` es:

```
vector = [1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0, 9.0, 10.0]

media = media(vector)
vector_centrado = []
for x in vector:
    vector_centrado.append(x - media)

vector_centrado
```

```
[-4.5, -3.5, -2.5, -1.5, -0.5, 0.5, 1.5, 2.5, 3.5, 4.5]
```

Si, en cambio, decidimos usar `map`, podemos hacer:

```
list(map(lambda x: x - media, vector))
```

```
[-4.5, -3.5, -2.5, -1.5, -0.5, 0.5, 1.5, 2.5, 3.5, 4.5]
```

Finalmente, se puede obtener el mismo resultado usando una *list comprehension*:

```
[x - media for x in vector]
```

```
[-4.5, -3.5, -2.5, -1.5, -0.5, 0.5, 1.5, 2.5, 3.5, 4.5]
```

La sintaxis general de una *list comprehension* que aplica una transformación sobre los elementos de un iterable es:

```
[<expresión> for elemento in iterable]
```

Como se observa en el ejemplo anterior, lo que aparece en la parte izquierda como `<expresión>` no tiene por qué ser una llamada a una función; puede ser **cualquier expresión válida** que produzca un resultado. Es decir, una operación matemática, un formateo de texto, la construcción de una estructura de datos, una llamada a una función, etc.

3.2 *Comprehension* como reemplazo de *filter*

Ahora veamos con mayor detalle cómo funciona una *list comprehension* que reemplaza al uso de `filter`. Para eso, retomemos el ejemplo de las palabras capicúa.

```
palabras = ["hola", "mate", "somos"," libro", "conocer", "anilina",
"programa"]
```

Inicialmente, podemos construir un listado de palabras capicúa usando un bucle `for`.

```
capicuas = []
for palabra in palabras:
    if palabra == palabra[::-1]:
        capicuas.append(palabra)
capicuas
```

```
['somos', 'anilina']
```

Luego, podemos construir el listado de palabras capicúa usando la función de orden superior `filter`.

```
list(filter(lambda x: x == x[::-1], palabras))
```

```
['somos', 'anilina']
```

Y finalmente, se puede obtener exactamente el mismo resultado mediante una *list comprehension*.

```
[palabra for palabra in palabras if palabra == palabra[::-1]]
```

```
['somos', 'anilina']
```

La sintaxis general de una *list comprehension* que filtra los elementos de un iterable es:

```
[elemento for elemento in iterable if <expresión_lógica>]
```

Al igual que en la *list comprehension* que aplica funciones a todos los ementos, `<expresión_lógica>` puede ser **cualquier expresión** de Python que devuelva un valor `True` o `False`, o que pueda interpretarse como tal.

También podría usarse una *list comprehension* que transforme elementos filtrados de un iterable:

```
[<expresión> for elemento in iterable if <expresión_lógica>]
```

Por ejemplo:

```
# Multiplica por 2 a los numeros impares de `range(5)`
[x * 2 for x in range(5) if x % 2]
```

```
[2, 6]
```

ℹ *Comprehensions* con expresiones condicionales 😱

La estructura general:

```
[elemento for elemento in iterable if <expresión_lógica>]
```

puede modificarse cuando se desea evaluar una expresión en caso de que se cumpla una condición y otra distinta si no se cumple. Para ello, se usa una expresión condicional directamente en la parte izquierda de la comprensión:

```
[<expresión_si_verdadero> if <condición> else <expresión_si_falso> for
elemento in iterable]
```

Por ejemplo:

```
numeros = [1, 2, 3, 4, 5]
[f"{x} es par" if x % 2 == 0 else f"{x} es impar" for x in numeros]
```

```
['1 es impar', '2 es par', '3 es impar', '4 es par', '5 es impar']
```

ℹ *Dictionary comprehensions* 😱 😱

Las comprensiones en Python no están limitadas a listas. Este patrón también puede utilizarse para construir otras estructuras de datos como diccionarios, conjuntos e incluso generadores (estructura que veremos más adelante).

Por ejemplo, una **comprensión de diccionario** permite crear un `dict` a partir de una secuencia de pares clave-valor:

```
def media(x):
    return sum(x) / len(x)

datos = [
    ("Marcos", (4, 8, 9, 9)),
    ("Joaquín", (10, 8, 8, 7)),
    ("Luján", (10, 9, 9, 10)),
]

{nombre: media(notas) for nombre, notas in datos}
```

```
{'Marcos': 7.5, 'Joaquín': 8.25, 'Luján': 9.5}
```

4 Evaluación parcial de funciones

En Fundamentos comenzamos a trabajar con *function factories*, es decir, con funciones que definen y devuelven funciones. El ejemplo que vimos consistía en la función `crear_multiplicador` que recibía un múltiplo y devolvía una función de un argumento que al llamarla realizaba la multiplcación. Así, era posible crear funciones para duplicar, triplicar, cuadruplicar, etc.

```
def crear_multiplicador(x):
    def interna(y):
        return y * x
    return interna

duplicar = crear_multiplicador(2)
triplicar = crear_multiplicador(3)

print(duplicar(10), triplicar(22))
```

```
20 66
```

Ahora bien, esta no es la única forma de crear funciones que multipliquen dos numeros dejando uno de sus argumentos fijo.

Una alternativa consiste en crear una función general de multiplicación y usar `partial` del módulo `functools` para obtener una versión de la misma con alguno de sus argumentos fijados.

```
def multiplicar(x, y):
    return x * y

multiplicar(7, 8)
```

```
56
```

```
from functools import partial

cuadruplicar = partial(multiplicar, 4)
cuadruplicar(2)
```

```
8
```

En esencia, `partial` toma una función y fija algunos de sus parámetros, devolviendo una nueva función con argumentos ya establecidos. Dicho de otro modo, `partial` produce una función parcialmente evaluada, de ahí su nombre.

De un modo similar, se podrían crear funciones de potencia a partir de una función genérica.

```
def potencia(x, n):
    return x ** n

cuadrado = partial(potencia, n=2)
cubo = partial(potencia, n=3)

print(cuadrado(5), cubo(9))
```

```
25 729
```

Mediante un ejemplo podemos ver que `partial` también permite fijar más de un parámetro. Supongamos que tenemos una lista de números que queremos estandarizar; es decir, restarles la media y dividir cada valor por el desvío.

```
nums = [
    4.74346239e-01, -2.90877176e-01, -1.44377789e+00, -4.48680759e+01,
    -1.21249801e+00, -3.32729317e-01,  2.21676912e-01,  1.05599711e+00,
    -3.62372053e+00, -2.96441579e-01, -4.28304222e+00,  1.55908820e+02,
    9.00858234e-01, -1.09384173e+00, -1.51083571e+00, -5.38491167e-01,
    -3.84153084e-02,  1.20393395e+00,  1.82651406e-01,  2.05179405e+00
]

def media(x):
    return sum(x) / len(x)

def varianza(x):
    numerador = 0
    x_media = media(x)
    for x_i in x:
        numerador += (x_i - x_media) ** 2
    return numerador / len(x)

estandarizar = partial(
    lambda x, media, desvio: (x - media) / desvio,
    media=media(nums),
    desvio=varianza(nums) ** 0.5
)
```

Línea 20

Definimos una función `lambda` que implementa la estandarización. Esta función recibe el valor a estandarizar, la media y el desvío correspondientes.

Líneas 21-22

Calculamos la media y el desvío de la lista, y luego los pasamos a `partial` como parámetros a fijar.

De esta manera, obtenemos la función `estandarizar`, que al recibir un número le resta la media y lo divide por el desvío calculado a partir de `nums`.

```
estandarizar(nums[0])
```

```
-0.12933243764138067
```

Y, si queremos estandarizar toda la secuencia, podemos usar una *list comprehension*.

```
[estandarizar(num) for num in nums]
```

```
[-0.12933243764138067,
 -0.1506204085674334,
 -0.18269328610772323,
 -1.390726359064761,
 -0.17625924440864513,
 -0.15178470535100874,
 -0.13636151853677025,
 -0.1131513242720763,
 -0.24333773925841856,
 -0.1507752062996549,
 -0.2616795995472022,
 4.1947440261328905,
 -0.11746717741646615,
 -0.1729583111265218,
 -0.1845587869464874,
 -0.1575088536123251,
 -0.1435970990449382,
 -0.10903582664474336,
 -0.13744718034588063,
 -0.08544896194045284]
```

ℹ Argumentos posicionales y nombrados 🔢 🏷️

`partial` puede utilizarse para fijar tanto argumentos posicionales como nombrados. Cuando recibe argumentos posicionales, estos se transmiten a la función original en el mismo orden; mientras que, si se le pasan argumentos nombrados, se reenvían como tales.

Por ejemplo, las siguientes llamadas a `partial` generan funciones equivalentes:

```
def prod(x, y):
    return x * y

partial(prod, 5)     # 5 * y
partial(prod, x=5)   # 5 * y
partial(prod, y=5)   # x * 5
```

5 Decoradores

En Ciudadanos de primera clase aprendimos que las funciones son un objeto como cualquier otro. Por eso, ya no nos sorprende que puedan pasarse como argumento a otra función o devolverse como resultado de otra función.

Ahora vamos a explorar un tipo de funciones que son muy útiles en Python: los decoradores.

Los decoradores son funciones que "envuelven" o "encapsulan" funciones y modifican su comportamiento.

Empecemos con un ejemplo: la función `decorador` recibe una función `fun`, define una función `envoltura` que contiene una llamada a `fun` y la devuelve.

```
def decorador(fun):

    def envoltura():
        print("Antes de llamar a la función...")
        fun()
        print("Listo, ya se llamó a la función.")

    return envoltura
```

Para mostrar el funcionamiento del decorador, definamos una función muy sencilla, que simplemente imprime un saludo.

```
def decir_hola():
    print("¡Hola hola!")

decir_hola()
```

```
¡Hola hola!
```

Ahora, invocamos a `decorador` pasandole la función `decir_hola` y obtenemos una nueva una función.

Podemos ver que esta nueva función es la función `envoltura` definida dentro del decorador.

```
nueva = decorador(decir_hola)
nueva
```

```
<function __main__.decorador.<locals>.envoltura()>
```

Antes de ejecutar la función `nueva`, intentemos anticipar qué va a ocurrir cuando la llamemos.

Al invocar `nueva`, se ejecutarán las siguientes tres líneas de código:

```
print("Antes de llamar a la función...")
fun()
print("Listo, ya se llamó a la función.")
```

Línea 1

La primera línea contiene directamente un `print`, por lo que podemos anticipar que lo primero que vamos a ver es un mensaje que dice `"Antes de llamar a la función..."`.

Línea 2

La segunda línea contiene una llamada a la función `fun`. Esta es la función que le pasamos a `decorador` al momento de crear `nueva`, es decir, es la función `decir_hola`.
Por lo tanto, habrá un segundo mensaje que dice `"¡Hola hola!"`.

Línea 3

Finalmente, se ejecuta la tercera línea, y como vemos que es un `print`, sabemos que vamos a ver un mensaje que dice `"Listo, ya se llamó a la función."`.

```
nueva()
```

```
Antes de llamar a la función...
¡Hola hola!
Listo, ya se llamó a la función.
```

En este ejemplo vemos que el decorador "envuelve" o "encapsula" a la función `decir_hola`. Gracias a esto, la función decorada ya no se ejecuta como antes, sino que ahora también imprime mensajes antes y después de realizar la tarea en su definición original.

5.1 Decoradores que reciben argumentos

Si intentamos pasarle argumentos a la función `nueva`, obtendremos un error. Este error no se debe a que la función decorada, `decir_hola`, no acepte parámetros, sino a que la función que devuelve el decorador, `envoltura`, no está preparada para recibirlos.

Ahora bien, si queremos que nuestra función de envoltura pueda transmitir argumentos a la función decorada, necesitamos un mecanismo flexible. No podemos conocer de antemano qué parámetros recibirá la función a decorar, justamente porque no sabemos cuál será esa función.

La solución es definir `envoltura` de manera que acepte una cantidad arbitraria de argumentos posicionales y nombrados. De esta forma, podemos propagar todos esos argumentos a la función decorada sin importar cuáles sean.

```
def decorador(fun):

    def envoltura(*args, **kwargs):
        if args:
            print("Argumentos posicionales:", args)
        if kwargs:
            print("Argumentos nombrados:", kwargs)
        fun(*args, **kwargs)

    return envoltura

def potencia(x, n):
    return x ** n

potencia = decorador(potencia)
```

Línea 3

`envoltura` recibe una cantidad arbitraria de argumentos posicionales y nombrados.

Línea 8

Cuando se llama a `fun`, se le pasan todos los argumentos posicionales y nombrados recibidos.

```
potencia(5, 3)
```

```
Argumentos posicionales: (5, 3)
```

```
potencia(5, n=3)
```

```
Argumentos posicionales: (5,)
Argumentos nombrados: {'n': 3}
```

```
potencia(x=5, n=3)
```

```
Argumentos nombrados: {'x': 5, 'n': 3}
```

El ejemplo muestra que el decorador imprime los argumentos de la función original, tanto posicionales como nombrados, siempre que se le haya pasado alguno.

5.2 Decoradores que devuelven valores

Si bien el decorador anterior funcionaba correctamente con funciones que reciben tanto argumentos posicionales como nombrados, no vemos que la función decorada devuelva la potencia calculada. Para que eso ocurra, la `envoltura` no solo tiene que llamar a `fun`, sino también retornar lo que esta retorne.

```
def decorador(fun):

    def envoltura(*args, **kwargs):
        if args:
            print("Argumentos posicionales:", args)
        if kwargs:
            print("Argumentos nombrados:", kwargs)
        return fun(*args, **kwargs)

    return envoltura

def potencia(x, n):
    return x ** n
```

Línea 8

Gracias a esta línea, la función `envoltura` retorna lo que sea que `fun` retorne.

```
potencia = decorador(potencia)
potencia(x=5, n=3)
```

```
Argumentos nombrados: {'x': 5, 'n': 3}
125
```

5.3 Azúcar sintáctico

Dado que los decoradores cumplen un rol muy importante en la programación con Python, el lenguaje ofrece una sintaxis especial para aplicarlos directamente al momento de definir una función.

Para ello, basta con escribir `@<nombre_decorador>` en la línea anterior a la definición de la función. Por ejemplo:

```
@decorador
def producto(x, y):
    return x * y

producto(3, 7)
```

```
Argumentos posicionales: (3, 7)
21
```

```
producto(x=3, y=7)
```

```
Argumentos nombrados: {'x': 3, 'y': 7}
21
```

De esta manera, no es necesario incluir líneas adicionales del estilo:

```
def funcion(...):
    ...
    return ...

funcion = decorador(funcion)
```

A este tipo de atajos sintácticos que brinda el lenguaje se los conoce como azúcar sintáctico (del inglés, *syntax sugar*).

5.4 Ejemplo: medir tiempo de ejecución

Hasta ahora, los ejemplos que vimos fueron un tanto artificiales, pensados únicamente para mostrar qué son los decoradores y cómo se utilizan. A continuación, presentamos un ejemplo más cercano a un uso práctico.

El decorador `timer` imprime el tiempo de ejecución que le toma a una función. Luego, lo aplicamos para comparar los tiempos entre la función *built-in* `max` y otra implementación que obtiene el máximo mediante una reducción.

```
import time

def timer(fun):
    def envoltura(*args, **kwargs):
        inicio = time.time()
        resultado = fun(*args, **kwargs)
        fin = time.time()
        print(f"{fun.__name__} demoró {fin - inicio:6f} segundos")
        return resultado
    return envoltura
```

```python
def mayor(x, y):
    if x > y:
        return x
    return y

@timer
def maximo_reduce(x):
    return reduce(mayor, x)

@timer
def maximo_builtin(x):
    return max(x)
```

```python
lista = list(range(1_000_000))

maximo_reduce(lista)
```

```
maximo_reduce demoró 0.044118 segundos
999999
```

```python
maximo_builtin(lista)
```

```
maximo_builtin demoró 0.017050 segundos
999999
```